

Object Oriented Programming in C++

C++ data types, operators, classes and
objects

Sushree Padhan

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

February 9, 2026

C++

A simple C++ program:

```
1 #include<iostream>
2 using namespace std;
3 int main() {
4     int i;
5     cout << "C++ is better than C!" <<endl;
6     cout<<"Enter an integer:"<<endl;
7     cin>>i;
8     cout <<"i="<<i<< endl;
9     return 0;
10 }
```

- **<iostream>**: Standard C++ header file used for input/output statements.
- **namespace** (a container for identifiers): defines a scope for the identifiers that are used in a program. **std** (namespace name) brings all the identifiers defined in std to the current global scope.
- The identifier **cin**: predefined object of istream class that represents the standard input stream.
- The identifier **cout**: predefined object of ostream class that represents the standard output stream.
- *cin >> i*: an input statement and causes the program to wait for the user to type in a number. >>: extraction operator used with cin.
- *cout << ...*: output statement to display. <<: insertion operator used with cout.

scanf() and printf()

- Origin: C (handle input and output)
- Not type-safe
- It does not support manipulators. It uses format specifiers.
- Often faster in heavy input output scenarios.

```

1  #include <cstdio>
2  int main() {
3      int m;
4      float n;
5      printf("Enter int and float: "
6             );
7      scanf("%d %f", &m, &n);
8      printf("You entered: %d and %f\n", m, n);
9      return 0; }

```

cin and cout

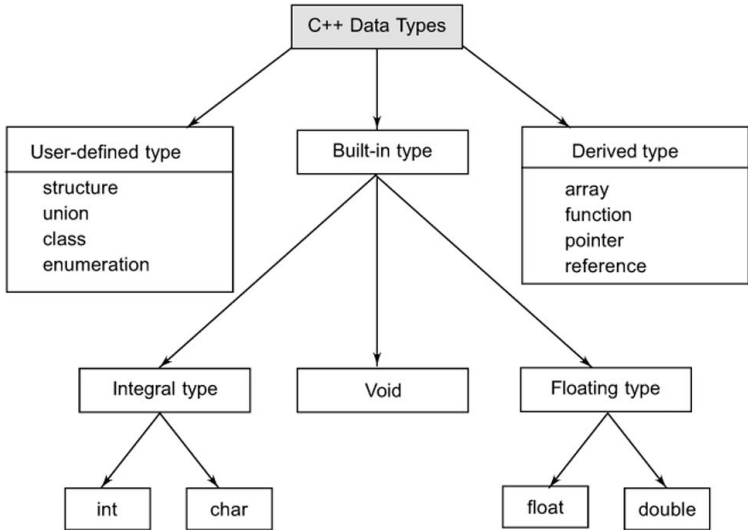
- Origin: C++ (handle input and output)
- Type-safe
- It supports manipulators
- Slightly slower due to type-safe and stream buffering.

```

1  #include <iostream>
2  using namespace std;
3  int main() {
4      int m;
5      float n;
6      cout << "Enter int and float: "
7             << endl;
8      cin >> m >> n;
9      cout << "You entered: " << m
10         << " and " << n << endl;
11     return 0; }

```

C++: Data Types



C++: User-defined Data Types

Arrays are used to group together similar type data elements. Structures are used for grouping elements with dissimilar types.

Structure:

```
1 struct book
2 {
3     char title[25];
4     char author[25];
5     int pages;
6     float price;
7 };
8 struct book book1, book2, book3;
```

- All members of a structure can be manipulated simultaneously.
- Structure members are allocated distinct memory locations.
- Structures in C++ behave just like a class. Almost everything that can be achieved with a class can also be done with structures.

C++: User-defined Data Types

Union: Size of a structure type is equal to the sum of the sizes of individual member types. Size of a union is equal to the size of its largest member element.

```
1 union result
2 {
3     int marks;
4     char grade;
5     float percent;
6 };
```

- The members of a union can be manipulated only one at a time.
- Union members share common memory space for their exclusive usage.
- Unions retain their core functionality in C++ with slight add-ons like declaration of anonymous unions.

C++: User-defined Data Types– Classes

- A **class** is a way to **bind the data and its associated functions** together. It allows the data (and functions) to be hidden.
- The class variables are known as objects, which are the central focus of OOP.
- General form of a class declaration:

```
1  class class_name
2  {
3      private:
4          variable declarations;
5          function declarations;
6      public:
7          variable declarations;
8          function declaration;
9  };
```

- Variables and functions are collectively called class members.
- The use of the keyword `private` is optional. By **default**, the **members** of a class are **private**.

C++: User-defined Data Types– Classes

- *private* can be accessed only from within the class. *public* members can be accessed from outside the class.
- Example:

```
1  class item
2      {
3          int number;    // variables declaration
4          float cost;    // private by default
5  public:
6          void getdata(int a, float b); // functions
           declaration
7          void putdata(void);
8      };
9      // ends with semicolon?
```

- function `getdata()` can be used to assign values to the member variables `number` and `cost`, and `putdata()` for displaying their values.

C++: User-defined Data Types– Classes

- **Creating objects:**

```
1  class item
2      {
3          ....
4          ....
5          ....
6      } x,y,z;
```

```
1  item x,y,z;
```

- objects x, y and z.

- **Accessing class members:**

```
1  object-name.function-name (actual-arguments);
2
3  x.getdata(100,75.5);
4
5  x.putdata();
```

C++: User-defined Data Types

Enumerated data type:

- Provides a way for attaching names to numbers, thereby increasing comprehensibility of the code.
- The enum keyword automatically enumerates a list of words by assigning them values 0,1,2, and so on. This facility provides an alternative means for creating symbolic constants.

Example:

```
1 enum shape{circle, square, triangle};  
2 enum colour{red, blue, green, yellow};  
3 shape ellipse;           // ellipse is of type shape  
4 // colour background;    // background is of type colour  
5 colour background = blue; // allowed  
6 colour background = 7;   // Error in C++
```

C++: Derived Data Types

Arrays:

```
1 #include <iostream>
2 using namespace std;
3 int main()
4 {
5     char s[4] = "xyz";
6     cout<<s;
7     return 0;
8 }
```

- Similar to that in C.

C++: Derived Data Types

Pointers:

- Pointers are declared and initialized as in C.
- Example:

```
1  int *ip,x;           // int pointer
2  ip = &x;             // address of x assigned to ip
3  *ip = 10;            // 10 assigned to x
```

- C++ adds the concept of constant pointer and pointer to a constant.
- Example:

```
1  int y=10, m=5;
2  int * const ptr1 = &y; // constant pointer (pointer
    // cannot move, data can change)
3  // ptr1=&m; //(not allowed) *ptr1=m; //(allowed)
4  int const * ptr2 = &m; // pointer to a constant
5  // address of m is assigned to ptr2. *ptr2=5 here. If m=6,
    // then *ptr2=6.
6  // *ptr2=y; //(not allowed, cannot modify value)
7  // ptr2=&y; //(allowed)
```

- We cannot modify the address that *ptr1* is initialized to. *ptr2* can point to any variable of correct type, but the contents of what it points to cannot be changed.

C++: Derived Data Types

Reference variables:

- A reference variable provides an alias for a previously defined variable.
- **Example:** If we make the variable `sum` a reference to the variable `total`, then `sum` and `total` can be used interchangeably to represent that variable.

```
1 data-type & reference-name = variable-name
2 float total = 100;
3 float &sum = total;
```

Reference

- A reference is another name for an existing variable.
- Once initialized, we can not bind it to a new variable.
- Initialization is mandatory (can not be null).
- No separate memory. Compiler treats it as same variable.

Pointer

- A pointer stores the address of a variable.
- It can point to somewhere else.
- Initialization is optional (can be null).
- Has its own memory.

C++: Derived Data Types

Example:

```
1  #include <iostream>
2  using namespace std;
3  int main() {
4  int a = 5;
5  int &ref = a;  // Reference variable (alias for a)
6  // int &ref; (Not allowed)
7  ref = 10;      // changes a
8  cout << "a after modifying reference: " << a << endl;  // 10
9  int *ptr = &a; // Pointer variable points to a
10 // int *ptr; (allowed) ptr = &a;
11 *ptr = 20;     // changes a via pointer
12 cout << "a after modifying pointer: " << a << endl; // 20
13 int b = 30;
14 ptr = &b;      // ptr points to b
15 *ptr = 50;     // changes b
16 cout << "b after modifying *ptr: " << b << endl; // 50
17 ref = b;       // this does not make ref refer to b, assigns b's
                  // value to a
18 cout << "a after ref = b: " << a << endl; // 50 (value assigned,
                  // still bind to a)
19 return 0;
20 }
```

C++: Derived Data Types– Functions

- Recall the C program syntax.

```
1 void show(); /* Function declaration */
2 int main()
3 {
4     . . . . .
5     show(); /* Function call */
6     . . . . .
7 }
8 void show() /* Function definition */
9 {
10     . . . . . /* Function body */
11 }
```

- In C++, the main() returns a value of type int to the operating system.

C++: Derived Data Types– Functions

- **Main function:** the **starting point for the execution** of a program.

```
1  int main()  
2  {  
3  . . . . .  
4  return 0;  
5  }
```

- **Function prototyping:**
A function declaration statement in the form:
type function-name (argument-list);
- Example: void show(int x, float y);
float volume(int x, float y, float z);

C++: Derived Data Types– Functions

- **Call by Reference:** permits to pass parameters to the functions by reference.
- When we pass arguments by reference, the ‘formal’ arguments in the called function become aliases to the ‘actual’ arguments in the calling function.

```
1 void swap(int &a, int &b) // a and b are reference variables
2 {
3     int t = a; // Dynamic initialization
4     a = b;
5     b = t;
6 }
```

- Function call:
swap(m, n);
will exchange the values of m and n using their aliases (reference variables), a and b.

C++: Derived Data Types– Functions

```
1  #include <iostream>
2  using namespace std;
3  // Call by value
4  int addValue(int a) {
5      a = a + 10;  // changes only local copy
6      return a; }
7  // Call by reference
8  int addReference(int &a) {
9      a = a + 10;  // changes original variable
10     return a; }
11 int main() {
12     int x = 20;
13     int y = 20;
14     int p=addValue(x);
15     cout << "After call by value, x = " << x <<"", p ="<<p<< endl;
16     int q=addReference(y);
17     cout << "After call by reference, y = " << y<<"", q ="<<q<< endl;
18     return 0; }
```

Output:

After call by value, x = 20, p =30

After call by reference, y = 30, q =30

C++: Derived Data Types– Functions

- **Return by Reference:** In C++, a function can also return a reference.

```
1  int& max(int &x, int &y)
2  {
3  if (x > y)
4  return x;
5  else
6  return y;
7  }
```

- max(a, b) will yield a reference to either a or b depending on their values.
- **Inline functions:** used to eliminate the cost of calls to small functions.

An inline function is expanded in line when it is invoked. Compiler replaces the function call with the corresponding function code.

C++: Derived Data Types– Functions

Example of return by reference:

```
1  int addValue(int a) {
2  a = a + 10;  // changes only local copy
3  return a;  // returns by value
4  }
5  int& addReference(int &a) {
6  a = a + 10;  // changes original variable
7  return a;  // returns reference to original variable
8  }
9  int main() {
10 int x = 20, y = 20;
11 int p=addValue(x);
12 cout << "After return by value, x = " << x <<"", p ="<<p<< endl;
13 int &q=addReference(y);
14 cout << "After return by reference, y = " << y<<"", q ="<<q<<
    endl;
15 q=50;
16 cout << "After modifying reference, y = " << y<<"", q ="<<q<<
    endl;
17 return 0; }
```

Output:

After return by value, x = 20, p =30

After return by reference, y = 30, q =30

After modifying reference, y = 50, q =50

C++: Derived Data Types– Functions

Inline Functions:

- Inline functions are defined as:

```
1  inline function-header
2  {
3  function body
4  }
```

- Example:

```
1  inline double cube(double a)
2  {
3  return(a*a*a);
4  }
```

- Inline functions invoked:

c = cube(3.0);

It may substitute 3*3*3 directly into the code.

d = cube(2.5+1.5);

On the execution, the values of c and d will be 27 and 64. If the arguments are expressions such as 2.5+1.5, the function passes the value of the expression, 4.

C++: Derived Data Types– Functions

- **Default arguments in functions:** default value is specified in a manner syntactically similar to a variable initialization.

```
1 float amount(float principal, int period, float rate=0.15);
```

- The above prototype declares a default value of 0.15 to the argument rate.

```
1 value=amount(5000,7);           // one argument missing
```

- Passes the value of 5000 to principal and 7 to period and then lets the function use default value of 0.15 for rate.
- value=amount(5000,5,0.12); Here no missing argument, so an explicit value of 0.12 is passed to rate.
- Only the **trailing arguments can have default values** and therefore we must **add defaults from right to left**.

C++: Derived Data Types– Functions

- An **argument** to a function can be declared as **const**, indicating that the **function should not modify the argument**.
- Examples:

```
1 #include <iostream>
2 using namespace std;
3 void show(const int
4     x) {
5     //x = 20; ///? ERROR:
6     //cannot modify a
7     //const parameter
8     cout << "Value of x="
9         << x << endl;
10    }
11 int main() {
12    show(10); //Value of
13    x= 10
14    return 0; }
```

```
1 #include <iostream>
2 using namespace std;
3 int length(const
4     char *p) {
5     // p[0] = 'A'; // ?
6     // ERROR: cannot
7     // modify const
8     // data
9     int count = 0;
10    while (*p != '\0') {
11        count++;
12        p++; }
13    return count; }
14 int main() {
15    cout << "Length= "<<
16        length("Hello")
17        << endl; //
18        Length= 5
19    return 0; }
```

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4 int length(const
5     string &s) {
6     // s = "Hello"; //
7     // ? ERROR: cannot
8     // modify a const
9     // reference
10    return s.length(); }
11 int main() {
12    string name = "Hello
13    C++";
14    cout << "Length= " <<
15        length(name) <<
16        endl; //Length=
17        9
18    return 0; }
```

C++: Derived Data Types– Functions

Function overloading:

- **Multiple functions** with **same function name** but with different argument lists.
- The function would perform different operations depending on the argument list in the function call. The **correct function** to be invoked is determined by **checking the number and type of the arguments** but not on the function type.
- Examples:

```
1 // Declarations
2 int add(int a, int b);           // prototype 1
3 int add(int a, int b, int c);    // prototype 2
4 double add(double x, double y);  // prototype 3
5 double add(int p, double q);     // prototype 4
6 double add(double p, int q);     // prototype 5
```

```
1 // Function calls
2 cout << add(5, 10);              // uses prototype 1
3 cout << add(15, 10.0);           // uses prototype 4
4 cout << add(12.5, 7.5);          // uses prototype 3
5 cout << add(5, 10, 15);          // uses prototype 2
6 cout << add(0.75, 5);            // uses prototype 5
```


C++: Derived Data Types– Functions

Function overloading:

- A function call first matches the prototype having the same number and type of arguments and then calls the appropriate function for execution. Selection involves:
 - ▶ The compiler first tries to find an **exact match** in which the types of actual arguments are the same, and use that function.
 - ▶ If an exact match is not found, the compiler uses the **promotions** to the actual arguments, such as

```
1 char to int  
2 short to int  
3 bool to int
```

to find a match.

- ▶ When either of them fails, the compiler tries to use the **built-in conversions** (like int to float, int to long, char to float) to the actual arguments and then uses the function whose match is unique.

C++: Derived Data Types– Functions

Function overloading:

- If the conversion is possible to have multiple matches, then the compiler will generate an error message.

```
1 long square(long n);  
2 double square(double x);
```

- A function call like

```
1 square(10);
```

will cause an error because the int argument can be converted to either long or double, making it ambiguous which square() method should be called.

- If none of these steps succeed, the compiler attempts user-defined conversions along with integral promotions and built-in conversions to find a unique match. User-defined conversions are commonly used in handling class objects.

C++: Derived Data Types– Functions

Function overloading:

```
1  #include <iostream>
2  using namespace std;
3
4  long square(long n);
5  double square(double x);
6  float add(int a, double b) {
7  return (a+b);}
8  float add(double a, int b) {
9  return (a+b);}
10 int main() {
11 //cout<<add(5,3)<<endl; //error: call of overloaded 'add(int,
    int)' is ambiguous
12 cout<<add(2.5,6)<<endl; //8.5
13 //cout<<square(10)<<endl; //error: call of overloaded 'square(
    int)' is ambiguous
14 cout<<square(5.6)<<endl; //31.36
15 return 0; }
16 long square(long n) {
17 return(n*n); }
18 double square(double n) {
19 return(n*n); }
```

C++: Operators

- All C operators are valid in C++.
- New operators:
 - ▶ :: (Scope resolution operator)
 - ▶ ::* (Pointer-to-member declarator: To declare a pointer to a member of a class)
 - ▶ ->* (Pointer-to-member operator: To access a member using a pointer to the object and a pointer to that member)
 - ▶ .* (Pointer-to-member operator: To access a member using object name and a pointer to that member)
 - ▶ new (Memory allocation operator)
 - ▶ delete (Memory release operator)
 - ▶ endl (Line feed operator)
 - ▶ setw (Field width operator)

C++: Operators

Scope resolution operator:

1

```
:: variable-name
```

- It allows access to the global version of a variable. For example, `::count` means the global version of the variable `count` (and not the local variable `count` declared in that block).
- It helps access to class variables or functions.
- To access namespace members.

C++: Operators

```
1  #include <iostream>
2  using namespace std;
3  int m = 10;                // global m
4  int main()
5  {
6      int m = 20;            // m redeclared, local to main
7
8      {
9          int k = m;
10         int m = 30;         // m declared again
11                             // local to inner block
12         cout << "inner block";
13         cout << "k = " << k << "\t";
14         cout << "m = " << m << "\t";
15         cout << "::m = " << ::m << "\t";
16     }
17     cout << "outer block \t";
18     cout << "m = " << m << "\t";
19     cout << "::m = " << ::m << "\t";
20     return 0;
21 }
```

O/p: inner block k = 20 m = 30 ::m = 10 outer block m = 20 ::m = 10

C++: Operators

Pointers to members:

- It is possible to take the address of a member of a class and assign it to a pointer.
- The address of a member can be obtained by applying the operator `&` to a class member name.
- A class member pointer can be declared using the operator `::*` with the class name.

```
1  class A {  
2  public:  
3  int m;  
4  void show() {  
5  cout<<"Hi"<<endl; }  
6  };  
7  int A ::* ip = &A :: m;  
8  void (A ::* fp)() = &A :: show;
```

- The *ip* pointer acts like a class member in that it must be invoked with a class object.
- `A::*` means "pointer-to-member of A class". `&A::m` means the "address of the m member of A class".

C++: Operators

Pointers to members:

- The pointer ip can now be used to access the member m inside member functions. Let "a" is an object of class "A" and a.m=10.

```
1 A a;  
2 cout << a.*ip; // display m  
3 (a.*fp)();
```

```
1 A* ap; // ap is pointer to object a  
2 ap = &a;  
3 cout << ap ->*ip; // display m  
4 cout << ap ->m; // same as above  
5 (ap ->*fp)();
```

- dereferencing operator ->* is used to access a member when we use pointers to both the object and the member.
- The dereferencing operator .* is used when the object itself is used with the member pointer.
- *ip is used like a member name.

C++: Operators

Pointers to members:

```
1  class M {
2  public:
3  int x; int y;
4  void set_xy(int a, int b) {
5  x = a; y = b; }
6  int sum(M m); };
7  int M::sum(M m) {
8  int M::* px = &M::x; //px is a pointer to a data member x of
    class M.
9  int M::* py = &M::y;
10 M *pm = &m;
11 int S = m.*px + pm->*py;
12 return S; }
13 int main() {
14 M n; //void (M::*pf[])()={&M::set_xy, &M::set_xyz}; multiple funs
15 void (M::*pf)(int,int) = &M::set_xy; //pf is a pointer to
    member function set_xy of class M.
16 (n.*pf)(10,20); //object and pointer to member
17 cout << "SUM = " << n.sum(n);
18 M *of = &n;
19 (of->*pf)(30,40); //pointer to object and pointer to member
20 cout << setw(10) << "SUM = " << n.sum(n) << endl;
21 return 0; }
```

Output: SUM = 30

SUM = 70

C++: Operators

Memory management operator:

- Although C++ supports malloc(), calloc() and free() functions, it also defines two unary operators **new** and **delete** that perform the task of allocating and freeing the memory in a better and easier way.

C Program

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 int main() {
4     int *p = (int *)malloc(
        sizeof(int));
5     *p = 5;
6     printf("%d", *p); //
        Prints: 5
7     free(p); }
```

C++ Program

```
1 #include <iostream>
2 #include <cstdlib>
3 using namespace std;
4 int main() {
5     int *p = (int *)malloc(
        sizeof(int));
6     *p = 5;
7     cout << *p; // Prints: 5
8     free(p); }
```

- new** operator can be used to **create objects of any type**, can also **initialize the memory**.

```
1 pointer-variable= new data-type;
2 int *p = new int; float *q = new float;
3 pointer-variable = new data-type(value);
4 int *p = new int(25);
5 float *q = new float(7.5);
```

C++: Operators

Memory management operator:

- When a data object is no longer needed, it is destroyed to release the memory space for reuse.
- **delete** operator can be used to free a memory, can delete the entire array pointed to by a pointer.

```
1 delete pointer-variable;  
2 delete p; delete q;  
3 delete [size] pointer-variable;  
4 delete[] p;
```

malloc() and free()

```
1 int main() {  
2 int *p = (int *)malloc(  
    sizeof(int));  
3 *p = 5;  
4 cout << *p; // Prints: 5  
5 free(p); }
```

new and delete

```
1 int main() {  
2 int *p = new int(10); //  
    allocate and  
    initialize  
3 *p = 5;  
4 cout << *p; // Prints: 5  
5 delete p; // we can also  
    write delete (p)  
6 return 0; }
```

C++: Operators

Memory management operator: Dynamically managed Arrays:-

```
1  #include <iostream>
2  using namespace std;
3  int main() {
4  int size = 5;
5  int *arr = new int[size]; // allocate array
6  for (int i = 0; i < size; i++) {
7  arr[i] = (i + 1) * 10; }
8  for (int i = 0; i < size; i++) {
9  cout << arr[i] << " "; } //10 20 30 40 50
10 cout << endl;
11 delete[] arr; // Deallocate the array arr
12 int** arr1 = new int*[n]; // dynamic array of int pointers
13 for (int i = 0; i < n; i++) {
14 arr1[i] = new int; // allocate memory for one int
15 *arr1[i] = (i + 1) * 10; }
16 cout << "Values in dynamic array of pointers:\n";
17 for (int i = 0; i < n; i++) {
18 cout << *arr1[i] << " "; } //10 20 30 40 50
19 cout << endl;
20 for (int i = 0; i < n; i++) {
21 delete arr1[i]; } // delete each int
22 delete[] arr1; // delete the array of pointers
23 return 0; }
```

C++: Operators

Manipulators:

- Manipulators are operators used with the insertion operator (<<) to modify—or manipulate—the way data is displayed.
- **endl** manipulator inserts a newline character and flushes the output buffer. It is commonly used for formatting output.

```
1  .....  
2  .....  
3  cout << m = << m << endl  
4  << n = << n << endl  
5  << p = << p << endl;  
6  .....  
7  .....
```

```
1  m = 2597  
2  n = 14  
3  p = 175
```

C++: Operators

Manipulators: setw()

- We can specify a common field width for all the numbers and force them to be printed right-justified.
- setw() applies only to the next item inserted into cout.

```
1 cout << setw(5) << sum << endl;
```

```
1 #include <iostream>
2 #include <iomanip>      // for setw
3 using namespace std;
4 int main()
5 {
6     int Basic = 950, Allowance = 95, Total = 1045;
7     cout << setw(10) << "Basic" << setw(10) << Basic <<
8         endl
9         << setw(10) << "Allowance" << setw(10) << Allowance <<
10        endl
11        << setw(10) << "Total" << setw(10) << Total << endl;
12    return 0;
13 }
```

```
1      Basic      950
2  Allowance      95
3      Total     1045
```

C++: Classes and Objects

- **Class:** A user-defined data type. A way to bind data and its associated functions together.

A template to create objects having similar properties. A blueprint for objects.

- A class allows data and functions to be hidden.

```
1  class class_name
2  {
3  private: //access specifier
4  variable declarations;
5  function declarations;
6  public: //access specifier
7  variable declarations;
8  function declaration;
9  };
```

- Variables and functions are collectively called class members.
- By default, the members of a class are private.
- private can be accessed only from within the class. public members can be accessed from outside the class.

C++: Classes and Objects

- Example:

```
1  class item
2  {
3  int number; // variables declaration
4  float cost; // private by default
5  public:
6  void getdata(int a, float b); // functions declaration
7  void putdata(void); // using prototype
8  };
9  // ends with semicolon?
```

- or

Forward declaration:

```
1  class item;
2
3  class item
4  {
5  ....
6  };
```


C++: Classes and Objects

- Difference between structure and class:

C++: Class

C: Structure

- ▶ keyword "struct"
- ▶ Group data
- ▶ All are public members and no access specifiers
- ▶ Not object-oriented
- ▶ Low data security

```
1  #include <stdio.h>
2  struct Student {
3  int id;
4  float marks; };
5  int main() {
6  struct Student s1;
7  s1.id = 101;
8  s1.marks = 85.5;
9  printf("ID: %d, Marks: %f\n",
10         s1.id, s1.marks);
11  return 0; }
```

- ▶ keyword "class"
- ▶ Group data and functions
- ▶ Access specifiers are there for all members and by default it is private
- ▶ Object-oriented
- ▶ High data security

```
1  #include <iostream>
2  using namespace std;
3  class Student {
4  public:
5  int id;
6  float marks; };
7  int main() {
8  Student s1;
9  s1.id = 101;
10 s1.marks = 85.5;
11 cout << "ID: " << s1.id << ",
12      Marks: " << s1.marks;
13 return 0; }
```

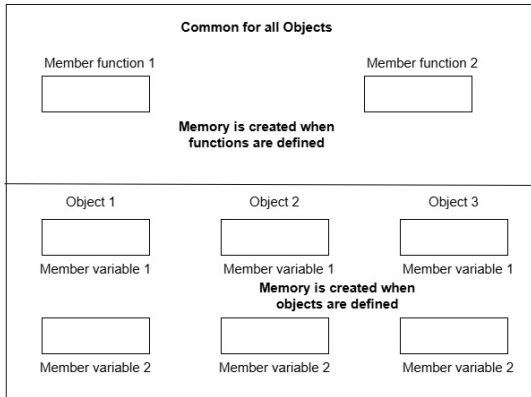
C++: Classes and Objects

- An **object** is an **instance** of a class.
- **Creating objects:**

```
1  class item
2  {
3  ....
4  } x,y;
5  item z;
```

- Objects x, y and z of item class. A class does not occupy memory unless an object is created.
- Member functions are created and placed in memory space only once when they are defined.
- All the objects belonging to that class use the same member functions, no separate space is allocated for member functions when the objects are created.
- Only space for member variables is allocated separately for each object. Separate memory locations for the objects are essential, because the member variables will hold different data values for different objects.

C++: Classes and Objects



- **Accessing class members:** by dot operator on the object

```
1 object-name.function-name (actual-arguments);  
2 x.getdata(100,75.5); //OK  
3 x.putdata(); //OK  
4 x.number=100; //illegal as number is declared as private
```

C++: Classes and Objects

Defining Member Functions:

- **Outside the class definition:**

Member function definition:

```
1  return-type class-name :: function-name (argument list)
2  {
3  Function body
4  }
```

Example:

```
1  void item :: getdata(int a, float b)
2  {
3  number = a;
4  cost = b;
5  }
6  void item :: putdata(void)
7  {
8  cout << "Number :" << number << "\n";
9  cout << "Cost :" << cost << "\n";
10 }
```

C++: Classes and Objects

Defining Member Functions:

- **Outside the class definition:**

Special characteristics:

- ▶ Several different classes can use the same function name. The 'membership label' will resolve their scope.
- ▶ Member functions can access the private data of the class. A nonmember function cannot do so.
- ▶ A member function can call another member function directly.

C++: Classes and Objects

Defining Member Functions:

- Inside the class definition:

```
1  class item
2  {
3  int number;
4  float cost;
5  public:
6  void getdata(int a, float b); // declaration
7  void putdata(void) // definition inside the class
8  {
9  cout << number << "\n";
10 cout << cost << "\n";
11 }
12 };
```

C++: Classes and Objects

This pointer:

- It is an implicit pointer and holds the address of currently calling object.
- "this" pointer is only accessible inside member functions.
- It is automatically provided by the compiler. `this->variable_name;`
- Why used?
 - Data and function parameters have same name.
 - Passing the current object to another function.
 - Return the current object.
 - Access object members explicitly.

```
1  class Student {  
2  int id;  
3  public:  
4  void set(int id) {  
5  this->id = id; }  
6  void display() {  
7  cout << "ID: " << id;} };  
8  int main() {  
9  Student s1;  
10 s1.set(101);  
11 s1.display();  
12 return 0;}
```

C++: Classes and Objects

This pointer:

```
1 class A {  
2     int x;  
3     public:  
4     void set(int val) {  
5         x = val;    // implicit use of this->x  
6     } };
```

```
1 class A {  
2     int x;  
3     public:  
4     void set(int x) {  
5         this->x = x;    // explicit use (parameter name same as data  
6         member)  
7     } };
```


C++: Classes and Objects

```
1  class A {
2  private:
3  int x;
4  public:
5  void setX(int x) {
6  this->x = x; }
7  // Passing the current object to another function
8  void sendObject(A obj) {
9  cout << "Sending object with x = ";
10 obj.display(); }
11 void passCurrentObject() {
12 sendObject(*this); // pass current object
13 }
14 A& addValue(int val) {
15 this->x += val;
16 return *this; //return the current obj by ref
17 }
18 void display() {
19 cout << "x = " << this->x << endl; } };
20 int main() {
21 A obj;
22 obj.setX(10);
23 obj.passCurrentObject();
24 obj.addValue(5).addValue(15).display();
25 return 0; }
```

C++: Classes and Objects

Making a function outside the class inline:

- A function defined inside a class is implicitly inline (no need to write "inline").
- A function defined outside the class must be explicitly marked with inline if we want inline behavior.

```
1  class item
2  {
3  ....
4  public:
5  void getdata(int a, float b); // declaration
6  };
7  inline void item :: getdata(int a, float b) // definition
8  {
9  number = a;
10 cost = b;
11 }
```

C++: Classes and Objects

Nesting of member functions:

- A member function of a class can be called only by an object of that class using a dot operator.
- A member function can be called by using its name inside another member function of the same class.

```
1  class Sample {  
2  private:  
3  int x;  
4  public:  
5  void set(int a) {  
6  x = a;}  
7  void show() {  
8  cout << "Value of x = " << x << endl; }  
9  void display() {  
10 show();    // nesting of member function  
11 } };  
12 int main() {  
13 Sample obj;  
14 obj.set(10);  
15 obj.display();  
16 return 0; }
```

C++: Classes and Objects

Private member functions:

- A private member function can only be called by another function that is a member of its class.
- Even an object cannot invoke a private function using the dot operator.

```
1 class sample {  
2     int m;  
3     void read(void); //private member function  
4     public:  
5     void update(void);  
6     void write(void);};
```

```
1 s1.read(); // won't work;  
2 // objects cannot access private members
```

```
1 void sample :: update(void)  
2 {  
3     read(); // simple call; no object used  
4 }
```

C++: Classes and Objects

Arrays within a class:

- Arrays can be used as member variables in a class.

```
1  #include <iostream>
2  using namespace std;
3  class Student {
4  private:
5  int marks[5];    // array inside the class
6  public:
7  void setMarks() {
8  for (int i = 0; i < 5; i++) {
9  cout << "Enter mark " << i + 1 << ": ";
10 cin >> marks[i]; }}
11 void displayMarks() {
12 cout << "Marks are: ";
13 for (int i = 0; i < 5; i++) {
14 cout << marks[i] << " "; }
15 cout << endl; }
16 };
17 int main() {
18 Student s;
19 s.setMarks();
20 s.displayMarks();
21 return 0; }
```

C++: Classes and Objects

Static data member:

- The properties of a **static** member variable are similar to that of a C static variable.
- Special characteristics:
 - ▶ It is initialized to zero when the first object of its class is created. No other initialization is permitted.
 - ▶ Only one copy of that member is created for the entire class and is **shared by all the objects of that class**, no matter how many objects are created.
 - ▶ It is visible only within the class, but its lifetime is the entire program. It is declared inside class but defined outside.
 - ▶ Static variables are normally used to maintain values common to the entire class. Example: a static data member can be used as a counter that records the occurrences of all the objects.

C++: Classes and Objects

Example of static data member:

```
1  class item {
2  static int count;
3  int number;
4  public:
5  void getdata(int a) {
6  number = a;
7  count++; }
8  void getcount(void){
9  cout << "count:";
10 cout << count << "\n"; } };
11 int item :: count; // definition of static member variable
12 int main() {
13 item a, b;
14 a.getcount (); b.getcount (); // display count
15 a.getdata(100); // getting data into object a
16 b.getdata(200); // getting data into object b
17 cout << " After readingdata" << "\n";
18 a.getcount(); b.getcount();
19 return 0; }
```

count: 0

count: 0

After reading data

count: 2

count: 2

C++: Classes and Objects

Static member function:

Properties:

- A static function can have **access to only other static members** (functions or data) of the same class.
- A static member function can be **called using the class name** (instead of its objects) as: `class-name :: function-name;`

```
1  class math {  
2  public:  
3  static int add(int a, int b) {    // static member function  
4  return a + b;  
5  } };  
6  int main() {  
7  int result = math::add(5, 3);    // call using class name  
8  cout << "Sum = " << result << endl;  
9  return 0;}
```

Output:

Sum = 8

C++: Classes and Objects

Arrays of Objects:

- We can have arrays of variables that are of the type class.

```
1  class employee {  
2  char name[30];  
3  float age;  
4  public:  
5  void getdata(void);  
6  void putdata(void); };
```

- employee is a class type and can be used to create objects that relate to different categories of the employees.
- Example:

```
1  employee manager[3]; // array of manager  
2  employee worker[75]; // array of worker
```

- The array manager contains three objects, manager[0], manager[1] and manager[2], of type employee class. We can use the usual array-accessing methods to access individual elements, and then . operator to access the member functions. **manager[i].putdata();**

C++: Classes and Objects

Objects as function arguments:

```
1 class Number {  
2 public:  
3 int value;  
4 };
```

```
1 // Function that takes an object as argument  
2 void show(Number n) {  
3 cout << "Value = " << n.value << endl;  
4 }  
5 int main() {  
6 Number obj,obj1;  
7 obj.value=10;  
8 show(obj);    // passing object as argument  
9 show(obj1);  
10 return 0;  
11 }
```

Output:

```
1 Value = 10  
2 Value = 0
```

C++: Classes and Objects

Friend functions:

- A nonmember function cannot have an access to the private data of a class. Sometimes, we need a function to operate on objects of different classes.
- A friend function is allowed to access the private data members of a class. Such a function need not be a member of any class.

```
1  class ABC
2  {
3  ....
4  public:
5  ....
6  ....
7  friend void xyz(void); // declaration preceded by friend
                             keyword
8  };
```

C++: Classes and Objects

Special characteristics of friend functions:

- It can be declared either in the public or the private part of a class without affecting its meaning. It is defined outside the class, not inside the main function.
- It has the objects as arguments.
- It is not in the scope of the class to which it has been declared as friend. Since it is not in the scope of the class, it cannot be called using the object of that class.
- It can be invoked like a normal function without the help of any object.
- It cannot access the member names directly and has to use an object name and dot membership operator with each member name.

C++: Classes and Objects

Example using friend functions:

```
1  class Numbers {
2  private:
3  int a, b;
4  public:
5  void set_xyvalues(int x, int y) {
6  a = x;
7  b = y; }
8  // Friend function declaration
9  friend float mean(Numbers n);
10 };
11 // Friend function definition
12 float mean(Numbers n) {
13 return (n.a + n.b) / 2.0;    // access private members
14 }
15 int main() {
16 Numbers n;
17 n.set_xyvalues(10, 20);
18 cout << "Mean value = " << mean(n) << endl;
19 return 0; }
```

Output:

Mean value = 15

C++: Classes and Objects

Example of two classes using friend functions:

```
1  class ABC; // Forward declaration
2  class XYZ {
3  int x;
4  public:
5  void setvalue(int i) {x= i;}
6  friend void max(XYZ, ABC); };
7  class ABC {
8  int a;
9  public:
10 void setvalue(int i) {a= i;}
11 friend void max(XYZ, ABC); };
12 void max(XYZ m, ABC n) // Definition of friend
13 {
14 if(m.x >= n.a) cout << m.x;
15 else
16 cout << n.a;
17 }
18 int main() {
19 ABC abc;
20 abc.setvalue(10);
21 XYZ xyz;
22 xyz.setvalue(20);
23 max(xyz, abc);
24 return 0;}
```

C++: Classes and Objects

Local classes:

- Classes can be defined and used inside a function or a block.

```
1 void test() // function
2 {
3     class student // local class
4     {
5     public:
6     void show() {
7         cout << "This is a local class" << endl; } };
8     student obj; // object of local class
9     obj.show(); }
10 int main() {
11     test();
12     return 0; }
```

- Local classes can use global variables and static variables declared inside the function. A local class is visible only within the function in which it is defined.
- Local classes cannot have static data members. Local classes cannot have friend functions declared outside the local class scope.
- The enclosing function cannot access the private members of a local class, and this restriction cannot be removed using friend.

Any Questions?